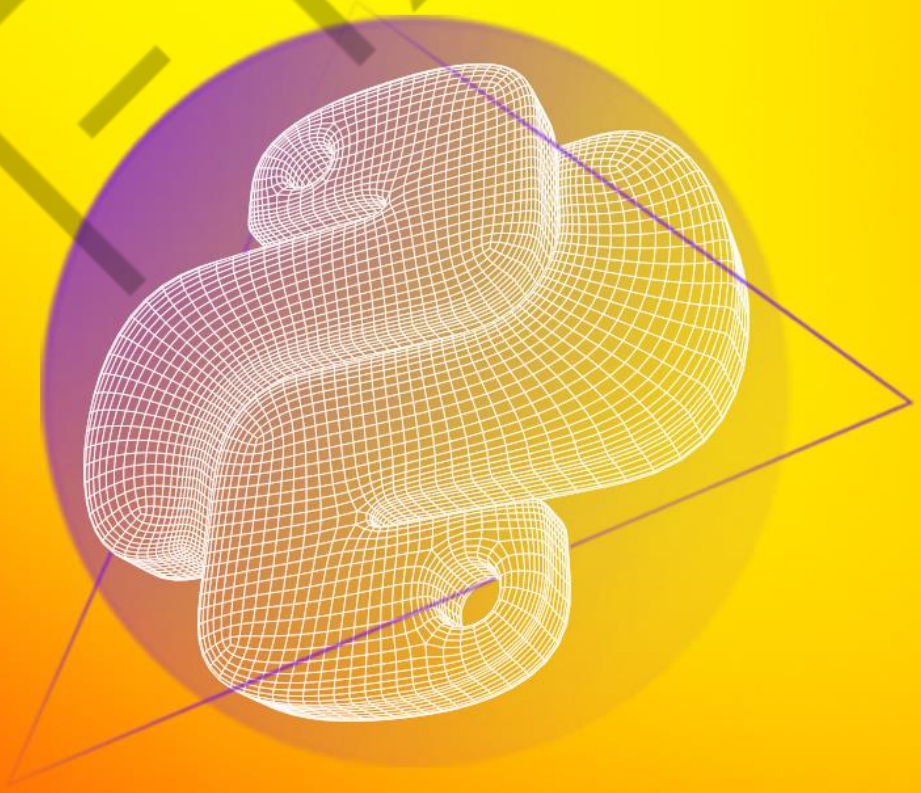
FIAP ON

DJANGO WEB FRAMEWORK

MODELOS E INTEGRAÇÕES

ANDREY MASIERO



2

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 2.1 – Criação do arquivo de migração do modelo para o banco de dados 6
Figura 2.2 – Execução do comando migrate para o modelo de livro.....7
Figura 2.3 – Aparição do modelo livro no site administrativo 8
Figura 2.4 – Formulário de inclusão de livro 8
Figura 2.5 – Inclusão do livro realizada com sucesso 9
Figura 2.6 – Tabela com quatro livros inclusos 9
Figura 2.7 – Tabela com a exibição dos livros ajustada.....10
Figura 2.8 – Tabela configurada com filtros, paginação, busca e campos com link de acesso 11
Figura 2.9 – Inclusão do modelo Autor na área administrativa.....12
Figura 2.10 – Autores cadastrados no sistema 13
Figura 2.11 – Erro da chave estrangeira 14
Figura 2.12 – Solicitação de valores padrão 14
Figura 2.13 – Autores aparecendo no cadastro do livro 15
Figura 2.14 – Exibição dos nomes dos autores no cadastro de livro 16

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 2.1 – Criação da classe Livro em models 5
Código-fonte 2.2 – Arquivo de migrations 7
Código-fonte 2.3 – Registrando o modelo na área administrativa do site 7
Código-fonte 2.4 – Classe ExibeLivro para personalizar a exibição do livro no site administrador 10
Código-fonte 2.5 – Classe ExibeLivro com as principais configurações realizadas 11
Código-fonte 2.6 – Classe modelo Autor.....11
Código-fonte 2.7 – Arquivo admin.py do app Autores 12
Código-fonte 2.8 – Inclusão da foreing key Autor.....14
Código-fonte 2.9 – Inclusão do método __str__ no modelo Autor.....16

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

2 MODELOS E INTEGRAÇÕES 5
2.1 Representando um livro 5
2.2 Alterando a exibição dos livros 9
2.3 Representando um autor 11
2.4 Autor e livro juntos novamente 13
REFERÊNCIAS 17

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 MODELOS E INTEGRAÇÕES

Após a instalação e configuração do projeto, inicia-se o trabalho de escrever os códigos referentes aos modelos ou classes que representam o negócio. Depois de configurarmos os modelos e deixarmos todos prontos para uso, poderemos partir para a parte gráfica. Vamos iniciar com o modelo da classe livro.

2.1 Representando um livro

O Django fornece uma abstração para representarmos os objetos referentes ao negócio do projeto em questão. Essa abstração é conhecida como modelo. Modelos são fonte única e definitiva de informações de dados. Eles são um molde que deve ser utilizado como base por todos os itens dizendo que todo livro, por exemplo, possui algumas características, como título, quantidade de páginas, editora, edição, descrição, idioma e ISBN.

Cada modelo Python criado dentro do Django deve herdar da classe models.Model e cada atributo representará uma coluna no banco de dados. A API fornecida pelo Django para conexão com o banco de dados SQLite fará o trabalho de conectar e já criar toda a estrutura do banco.

Para iniciar a criação do modelo de livro, vamos acessar o arquivo models.py disponível na pasta do app livros. No arquivo vamos criar a classe Livro recebendo models.Model como parâmetro. Essa é a forma de herança do Python. Veja o código-fonte “Criação da classe Livro em models”.

```
from django.db import models

class Livro(models.Model):
    titulo = models.CharField(max_length=200)
    descricao = models.TextField()
    paginas = models.IntegerField()
    editora = models.CharField(max_length=50)
    edicao = models.IntegerField()
    ano_publicacao = models.IntegerField()
    idioma = models.CharField(max_length=20)
    isbn = models.CharField(max_length=20)
```

Código-fonte 2.1 – Criação da classe Livro em models
Fonte: Elaborado pelo autor

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No código-fonte “Criação da classe Livro em models” atribuímos a cada um dos atributos o resultado de uma função do módulo models.

- CharField():contém vários argumentos a serem passados para modularizar da forma como precisamos, e para limitar a escrita do campo informamos a capacidade máxima de caracteres através da propriedade **max_lenght.**
- TextField(): uma caixa de texto para uma maior quantidade de palavras.
- IntegerField(): para números inteiros.

Após a criação do modelo precisamos refletir isso no banco de dados. Para que isso ocorra, utilizaremos o comando makemigrations. No terminal digite python manage.py makemigrations. O resultado deve ser parecido com a figura “Criação do arquivo de migração do modelo para o banco de dados”.

```
D:\django\library-app\library (main -> origin)
(venv)   python manage.py makemigrations
Migrations for 'livros':
  livros\migrations\0001_initial.py
    - Create model Livro
```

Figura 2.1 – Criação do arquivo de migração do modelo para o banco de dados
Fonte: Elaborado pelo autor

Um arquivo foi criado no diretório migrations com o nome de 0001_initial.py. Se observarmos o conteúdo apresentado são todas as configurações para o banco de dados, conforme configurado via modelo. Veja o código-fonte “Arquivo de migrations”.

```
from django.db import migrations, models


class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

    operations = [
        migrations.CreateModel(
            name='Livro',
            fields=[
                ('id', models.AutoField(auto_created=True,
primary_key=True, serialize=False, verbose_name='ID')),
                ('titulo', models.CharField(max_length=200)),
                ('descricao', models.TextField()),
                ('paginas', models.IntegerField()),
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                    ('editora', models.CharField(max_length=50)),
                    ('edicao', models.IntegerField()),
                    ('ano_publicacao', models.IntegerField()),
                    ('idioma', models.CharField(max_length=20)),
                    ('isbn', models.CharField(max_length=20)),
                ],
            ),
    ]
```

Código-fonte 2.2 – Arquivo de migrations
Fonte: Elaborado pelo autor

A efetivação da criação da tabela no banco de dados e a construção do modelo na área de administração é realizada através do comando migrate, como já fizemos anteriormente. No terminal execute o comando python manage.py migrate, conforme a figura “Execução do comando migrate para o modelo de livro”.

```
D:\django\library-app\library (main -> origin)
(venv)   python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, livros, sessions
Running migrations:
  Applying livros.0001_initial... OK
```

Figura 2.2 – Execução do comando migrate para o modelo de livro
Fonte: Elaborado pelo autor

Para acessarmos nosso modelo e gerenciar seu conteúdo pela área de administração, precisamos registrá-lo através do arquivo admin.py. Acesse o arquivo e insira o código de acordo com o código-fonte “Registrando o modelo na área administrativa do site”.

```
from django.contrib import admin
from .models import Livro

admin.site.register(Livro)
```

Código-fonte 2.3 – Registrando o modelo na área administrativa do site
Fonte: Elaborado pelo autor

Com o modelo registrado, basta acessar o site de administrator e validar a aparição dos livros no site. Confira o resultado na figura “Aparição do modelo livro no site administrativo”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 2.3 – Aparição do modelo livro no site administrativo
Fonte: Elaborado pelo autor

Agora com o modelo disponível no site do administrador, podemos cadastrar livros direto no banco de dados. Clique na opção adicionar e em seguida aparecerá um formulário para inclusão de um novo livro, como apresentado na figura “Formulário de inclusão de livro”.

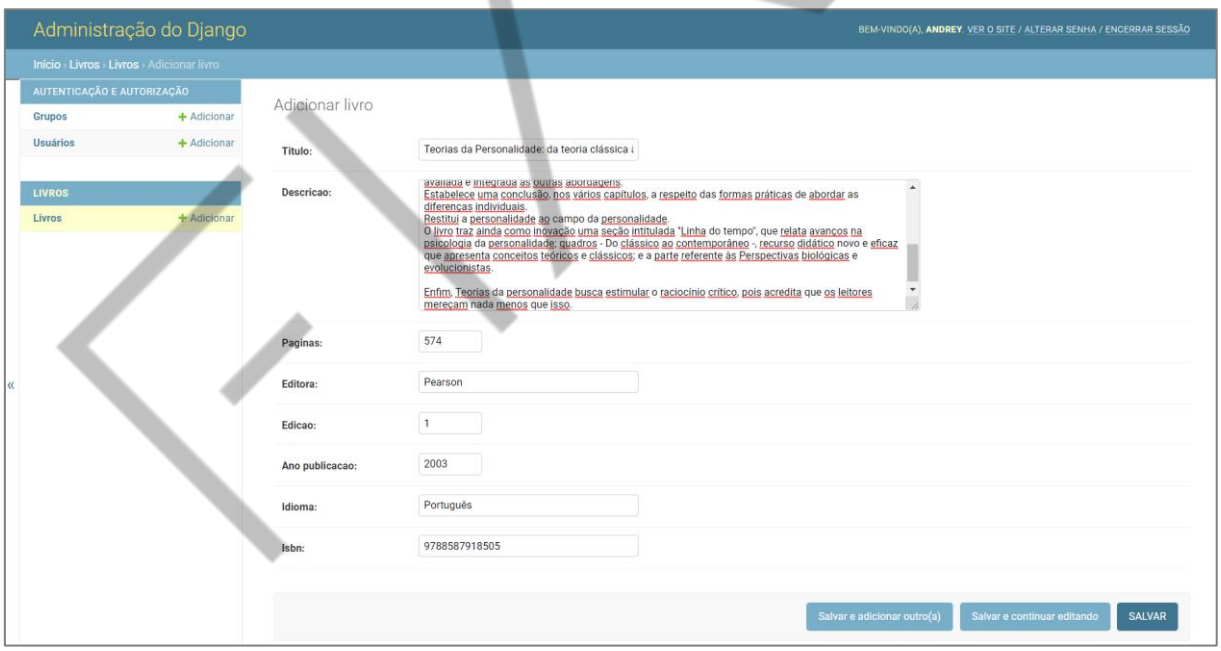


Figura 2.4 – Formulário de inclusão de livro
Fonte: Elaborado pelo autor

Preenchidos todos os campos, clique na opção salvar. O resultado obtido deve ser parecido com o apresentado na figura “Inclusão do livro realizada com sucesso”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

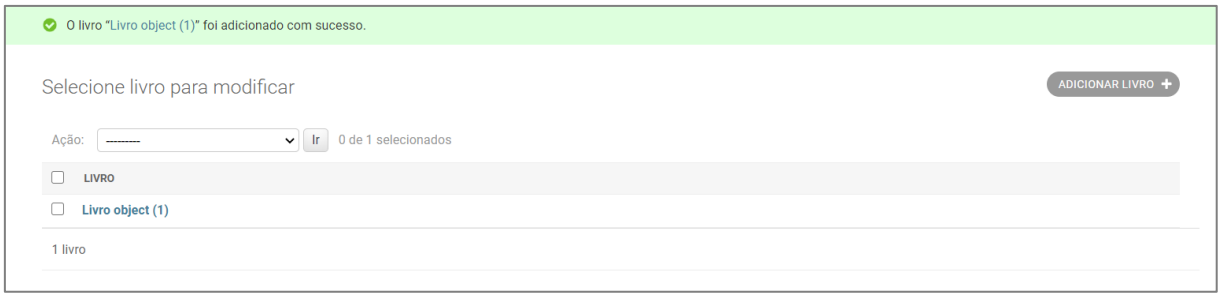


Figura 2.5 – Inclusão do livro realizada com sucesso
Fonte: Elaborado pelo autor

Vamos incluir mais 3 livros pelo menos. O resultado da tabela deve ser o mesmo que o apresentado na figura “Tabela com quatro livros inclusos”.

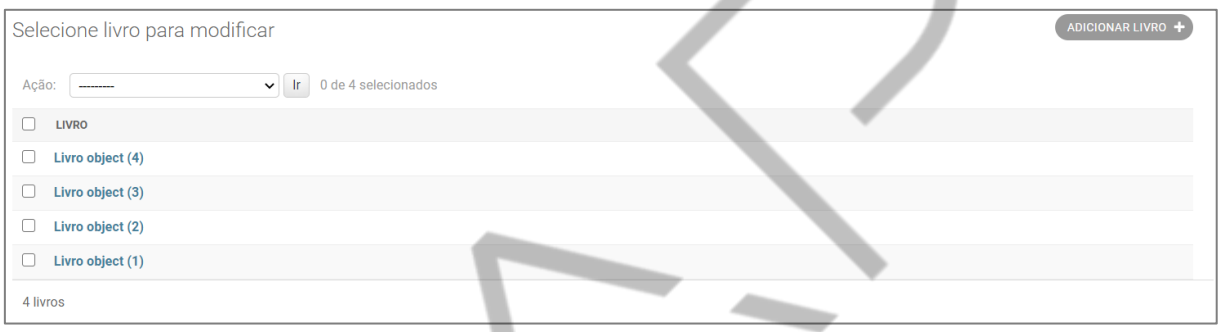


Figura 2.6 – Tabela com quatro livros inclusos
Fonte: Elaborado pelo autor

Perceba que a descrição do livro está meio estranha. Aparece “Livro object (1)” e não os dados dos livros. Será que é possível melhorar a exibição dessas informações? A resposta é sim, e vamos abordar isso na próxima seção.

2.2 Alterando a exibição dos livros

A exibição dos livros na tabela apresentada na figura “Tabela com quatro livros inclusos” não está representando uma boa experiência ao usuário. Por mais que seja na parte administrativa do site, isso não quer dizer que podemos trabalhar de qualquer jeito. O administrator também é um usuário e precisamos manter uma boa experiência para ele também. Para melhorar a apresentação da tabela de livros, necessitamos criar uma classe dentro do arquivo admin.py. Essa classe conterá a exibição dos livros de acordo com a nossa configuração. Veja o código-fonte “Classe ExibeLivro para personalizar a exibição do livro no site administrador”, a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from django.contrib import admin
from .models import Livro


class ExibeLivro(admin.ModelAdmin):
    list_display = ('id', 'titulo', 'ano_publicacao', 'isbn')


admin.site.register(Livro, ExibeLivro)
```

Código-fonte 2.4 – Classe ExibeLivro para personalizar a exibição do livro no site administrador
Fonte: Elaborado pelo autor

A classe ExibeLivro recebe como parâmetro o modelo admin.ModelAdmin. Ele nos permitirá utilizar a propriedade list_display, que recebe na atribuição uma tupla com todos os campos que desejamos exibir do livro. Selecionamos o id, título, ano de publicação e o ISBN. E, por fim, é necessário passar a classe ExibeLivro como parâmetro da admin.site.register, logo após a classe Livro. O resultado é apresentado na figura “Tabela com a exibição dos livros ajustada”.

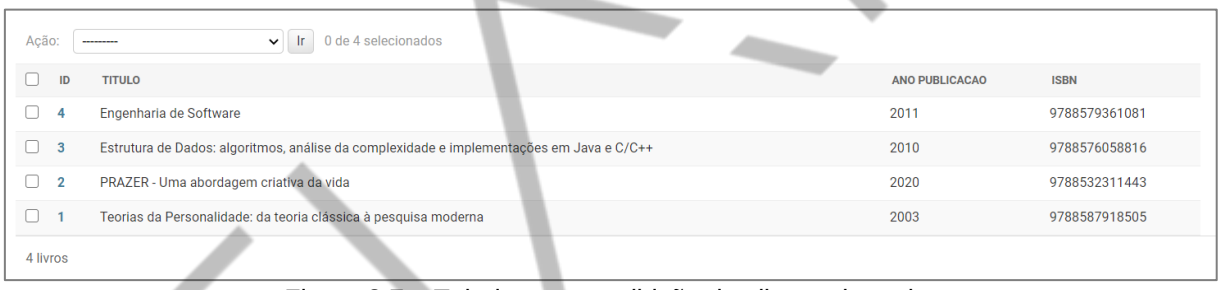


Figura 2.7 – Tabela com a exibição dos livros ajustada
Fonte: Elaborado pelo autor

Existem outras propriedades que podemos configurar através da classe ExibeLivro, as principais são:

- list_display_link: transforma os campos das colunas citadas em link de acesso ao objeto.
- search_fields: habilita a busca por um determinado campo ou conjunto de campos do objeto, por exemplo, o título do livro.
- list_filter: cria um filtro com base na tupla informando os campos que desejam ser considerados na filtragem, por exemplo, ano de publicação.
- list_per_page: cria a paginação de maneira automática com base no número definido de objetos por página, aqui informado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Aplicando esses atributos na classe ExibeLivro, temos o código-fonte “Classe ExibeLivro com as principais configurações realizadas”, exibido na sequência.

```
class ExibeLivro(admin.ModelAdmin):
    list_display = ('id', 'titulo', 'ano_publicacao', 'isbn')
    list_display_links = ('id', 'titulo')
    search_fields = ('titulo',)
    list_filter = ('ano_publicacao',)
    list_per_page = 2
```

Código-fonte 2.5 – Classe ExibeLivro com as principais configurações realizadas
Fonte: Elaborado pelo autor

O resultado da configuração apresentada através do código-fonte “Classe ExibeLivro com as principais configurações realizadas” é exibida através da figura “Tabela configurada com filtros, paginação, busca e campos com link de acesso”.

Figura 2.8 – Tabela configurada com filtros, paginação, busca e campos com link de acesso
Fonte: Elaborado pelo autor

Agora que configuramos a parte do livro, precisamos fazer a mesma coisa para o autor.

2.3 Representando um autor

O primeiro passo para criarmos a representação do autor em nosso sistema é através da classe modelo, que deve ser adicionada no app de autores. Confira o modelo através do código-fonte “Classe modelo Autor”.

```
from django.db import models


class Autor(models.Model):
    nome = models.CharField(max_length=200)
    nacionalidade = models.CharField(max_length=200)
```

Código-fonte 2.6 – Classe modelo Autor
Fonte: Elaborado pelo autor

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Essa é uma classe simples, com apenas o nome e nacionalidade do autor. Com o modelo criado, é necessário fazer o processo de makemigrations, para criar o arquivo de migração do modelo para o banco de dados e na sequência executar o migrate para efetivar a migração.

Logo após o processo de migração, vamos criar a classe de exibição do autor no arquivo admin.py e efetuar o registro do modelo e da classe de exibição para termos acesso no site de administração. O código-fonte “Arquivo admin.py do app Autores” apresenta a classe ExibeAutor e o registro do modelo para ser apresentado no site administrativo.

```
from django.contrib import admin
from .models import Autor


class ExibeAutor(admin.ModelAdmin):
    list_display = ('id', 'nome', 'nacionalidade')
    list_display_links = ('id', 'nome')
    search_fields = ('nome',)
    list_filter = ('nacionalidade',)
    list_per_page = 2


admin.site.register(Autor, ExibeAutor)
```

Código-fonte 2.7 – Arquivo admin.py do app Autores
Fonte: Elaborado pelo autor

Agora que concluímos o processo de criação e registro do modelo no site administrativo, devemos obter o resultado apresentado na figura “Inclusão do modelo Autor na área administrativa”, ao acessarmos a página inicial dessa área.

Figura 2.9 – Inclusão do modelo Autor na área administrativa
Fonte: Elaborado pelo autor

Após a inclusão dos autores no sistema, devemos obter o seguinte resultado, conforme apresentado na figura “Autores cadastrados no sistema”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

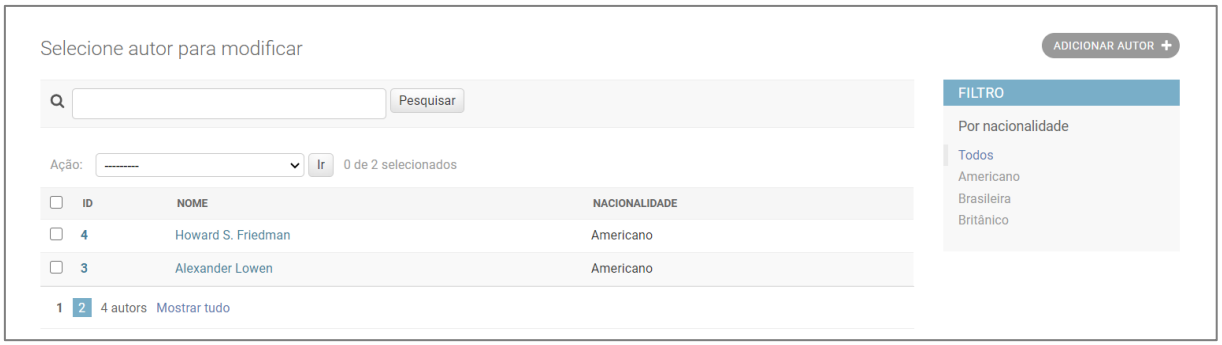


Figura 2.10 – Autores cadastrados no sistema
Fonte: Elaborado pelo autor

Com os dois modelos cadastrados, precisamos agora criar a integração entre ambos. Assim, podemos vincular o autor com seu respectivo livro.

2.4 Autor e livro juntos novamente

Cada livro criado em nosso sistema tem um autor. Dessa forma, vamos criar esse vínculo através de uma propriedade que adicionaremos no modelo de Livro. Para que essa operação ocorra com sucesso, devemos dizer que a propriedade autor tem um relacionamento de chave estrangeira com a classe autor. Isso tudo é possível apenas atribuindo o resultado da função ForeignKey de models à propriedade desejada.

A função ForeignKey recebe como parâmetro a classe que ela deve representar e um atributo chamado on_delete, que indica o comportamento que o código deve ter quando o registro for excluído. Nesse caso vamos colocar o comportamento CASCADE, que limpará os relacionamentos entre as classes de todas as tabelas de maneira automática. O código-fonte “Inclusão da foreing key Autor” apresenta o resultado da classe Livro após a inclusão da nova propriedade.

```
from django.db import models
from autores.models import Autor


class Livro(models.Model):
    autor = models.ForeignKey(Autor,
on_delete=models.CASCADE)
    titulo = models.CharField(max_length=200)
    descricao = models.TextField()
    paginas = models.IntegerField()
    editora = models.CharField(max_length=50)
    edicao = models.IntegerField()
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    ano_publicacao = models.IntegerField()
    idioma = models.CharField(max_length=20)
    isbn = models.CharField(max_length=20)
```

Código-fonte 2.8 – Inclusão da foreing key Autor
Fonte: Elaborado pelo autor

Mas ao tentar acessar os livros no site, obtemos o erro apresentado na figura “Erro da chave estrangeira”.

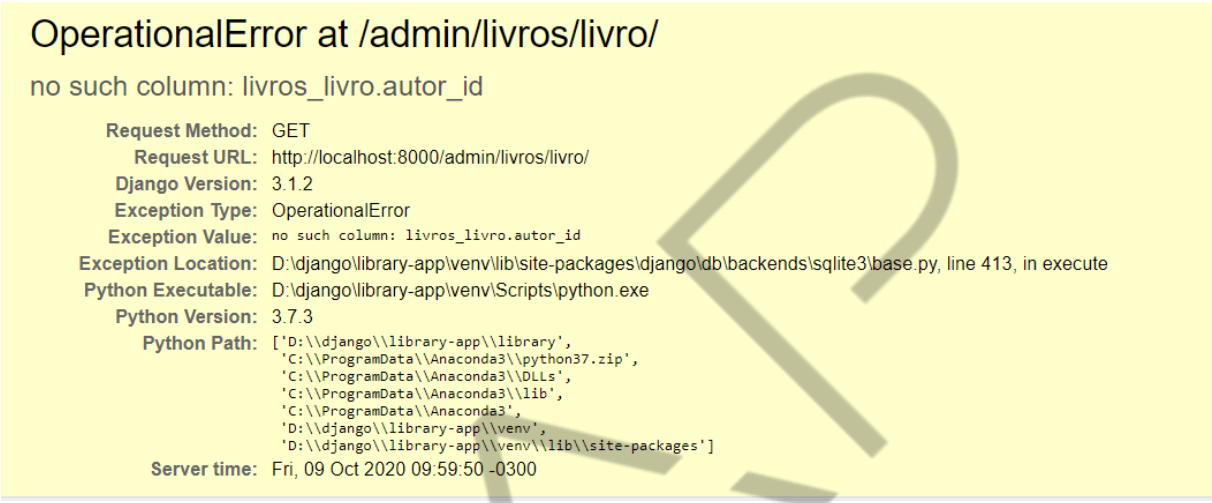


Figura 2.11 – Erro da chave estrangeira
Fonte: Elaborado pelo autor

Esse erro aconteceu por termos livros cadastrados previamente sem os autores. Para corrigir, devemos comentar a linha do relacionamento com o autor, assim acessaremos os livros para excluí-los, antes de fazer o cadastro.

Após remover os livros, remova o comentário da propriedade do autor. Para efetivar esse vínculo no banco de dados, vamos precisar executar os comandos makemigrations e migrate, novamente.

Ao executar o comando makemigrations obtemos uma mensagem, conforme figura “Solicitação de valores padrões”.

```
D:\django\library-app\library (main -> origin)
(venv)   python manage.py makemigrations
You are trying to add a non-nullable field 'autor' to livro without a default; we ca
n't do that (the database needs something to populate existing rows).
Please select a fix:
 1) Provide a one-off default now (will be set on all existing rows with a null valu
e for this column)
 2) Quit, and let me add a default in models.py
Select an option: |
```

Figura 2.12 – Solicitação de valores padrão
Fonte: Elaborado pelo autor

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A mensagem significa que estamos tentando preencher alguns campos que possuem algumas linhas. Para solucionar, temos duas opções ofertadas pelo sistema:

1) podemos prover um valor default agora; ou

2) podemos adicionar este valor em models.py.

Vamos trabalhar com a primeira opção digitando 1 como resposta ao terminal. Em seguida, definimos uma String vazia com '' (aspas simples) para este valor default, fazendo com que seja gerada a migração de livro. Agora podemos fazer a migração de maneira efetiva.

Nesse momento, se acessarmos o nosso sistema para cadastrar um livro, podemos ter acesso a um combo com os autores cadastrados. Veja a figura “Autores aparecendo no cadastro do livro”, exibindo o combo com as opções recuperadas do banco de dados.

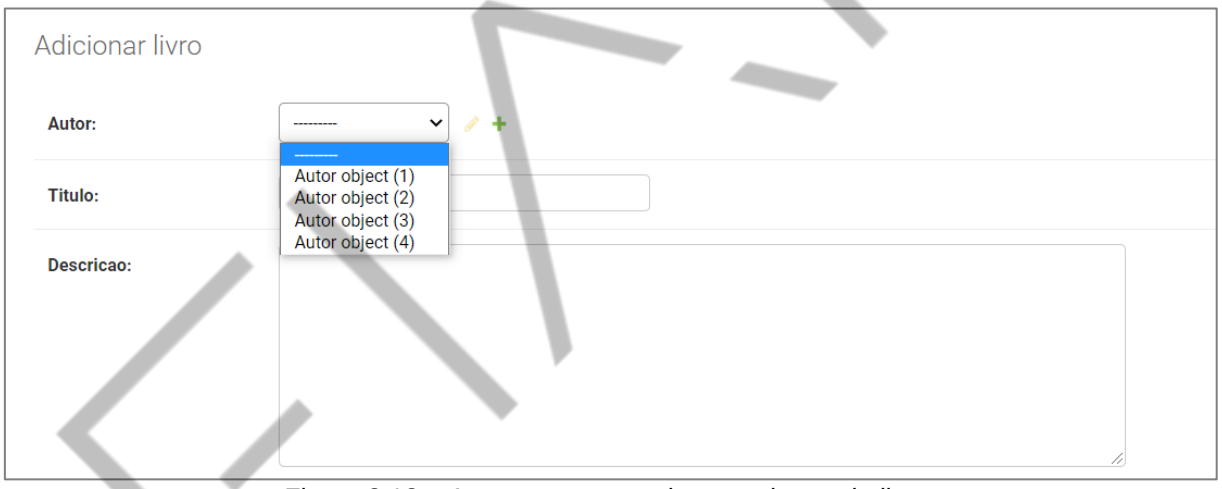


Figura 2.13 – Autores aparecendo no cadastro do livro
Fonte: Elaborado pelo autor

Contudo, a informação exibida não ajuda muito a saber qual é o autor disponível para seleção. Isso ocorre porque o Python não sabe traduzir as propriedades de uma classe em um formato legível para nós, seres humanos. Para que essa tradução seja realizada de maneira adequada, precisamos ajudar o Python. A ajuda vem através a implementação do método __str__ dentro da classe. Com esse método, podemos indicar ao Python quais propriedades vamos exibir quando exibirmos o objeto. No caso do autor, o legal é apresentar o nome dele. Vamos modificar a classe Autor de acordo com o código-fonte “ Inclusão do método __str__ no modelo Autor”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class Autor(models.Model):
    nome = models.CharField(max_length=200)
    nacionalidade = models.CharField(max_length=200)

    def __str__(self):
        return self.nome
```

Código-fonte 2.9 – Inclusão do método __str__ no modelo Autor
Fonte: Elaborado pelo autor

O método __str__ recebe como parâmetro self que representa a instância do objeto Autor. Assim, é possível conseguir acessar os demais recursos da classe. Esse método deve retornar uma string, então utilizamos o próprio nome para essa implementação, que já atende as regras de negócios planejadas. Confira na figura "Exibição dos nomes dos autores no cadastro de livro" o resultado da implementação do método __str__ no modelo Autor.

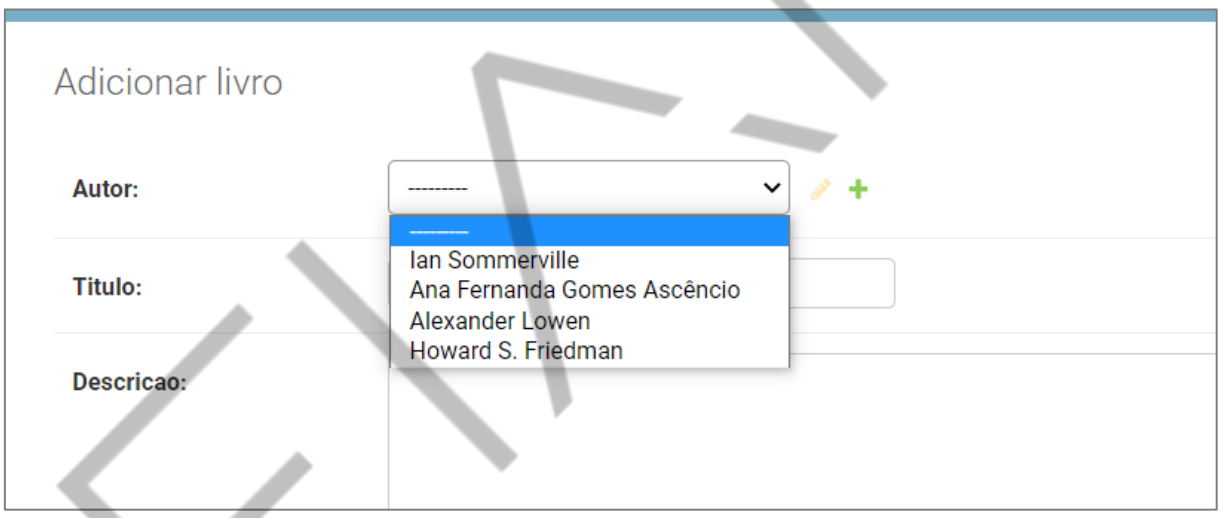


Figura 2.14 – Exibição dos nomes dos autores no cadastro de livro
Fonte: Elaborado pelo autor

Pronto, agora os modelos foram criados e já estão vinculados da maneira desejada. Precisamos agora começar a construir nosso front-end, mas isso é assunto para outro capítulo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

Django. **Documentação do Django Web Framework**. [s.d.]. Disponível em: https://www.djangoproject.com/. Acesso em: 05 out. 2020.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com